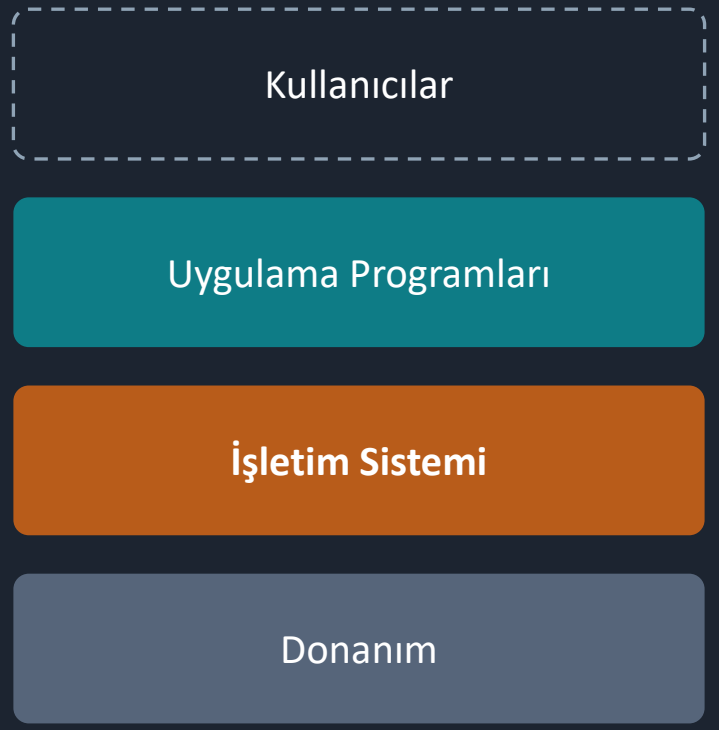
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 1

İşletim Sistemlerine Giriş ve Bilgisayar Sistemi Organizasyonu



Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

İşletim sistemi nedir?

Donanım ile programlar arasında neden bir aracı gerekir?

2

Tarihsel gelişim

Hangi sorunlar yeni İS yaklaşımlarını doğurdu?

3

Sistem organizasyonu

Kesmeler, depolama hiyerarşisi ve G/Ç nasıl çalışır?

4

Sistem mimarileri

Tek işlemciden kümelere: ne değişir?

5

Çoklu programlama ve zaman paylaşımı

CPU neden boş kalmamalı?

6

Koruma mekanizmaları

İkili mod ve Donanım Zamanlayıcısı neden gerekli?

7

İS'nin yönetim alanları

Dönem boyunca neleri öğreneceğiz?

8

Hesaplama ortamları ve açık kaynak

İS'ler hangi ortamlarda çalışır?

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. İşletim sisteminin rolünü kullanıcı ve sistem bakış açılarıyla açıklar
2. İşletim sistemini genişletilmiş makine ve kaynak yöneticisi olarak yorumlar
3. Kesme tabanlı çalışmayı adım adım anlatır; kesme ile tuzağı ayırt eder
4. Depolama hiyerarşisini hız, kapasite ve maliyet ödünleşimiyle açıklar
5. Çoklu programlama ile zaman paylaşımını ayırt eder
6. İkili modun, ayrıcalıklı komutların ve Donanım Zamanlayıcısının koruma amacını açıklar

Bu Konu Neden Önemli?

GÜNLÜK EYLEM	ARKA PLANDA İŞLETİM SİSTEMİNİN YAPTIĞI İŞ
Klavyede bir tuşa basmak	Aygıttan gelen kesme işlenir; karakter doğru programa iletilir
Aynı anda müzik dinleyip kod yazmak	CPU, programlar arasında fark edilmeyecek hızda paylaştırılır
Bir dosyayı kaydetmek	Veri, bellekten kalıcı depolamaya güvenli biçimde yazılır

Bu ders, kullanıcının çoğu zaman görmediği bu katmanın — işletim sisteminin — nasıl çalıştığını inceler.

Bilgisayar Sisteminin Dört Bileşeni

Kullanıcılar
insanlar, diğer makineler

Uygulama Programları
derleyici · web tarayıcısı · metin düzenleyici · veritabanı

İşletim Sistemi
donanımın kullanımını denetler ve eşgüdümler

Donanım
CPU · bellek · G/Ç aygıtları

Kaynak: Silberschatz vd., Bölüm 1.1

Donanım temel hesaplama kaynaklarını sağlar

Uygulamalar bu kaynakları kullanıcı problemlerini çözmek için kullanır

İşletim sistemi kaynakların uygulamalar ve kullanıcılar arasında kullanımını denetler

Akılda tutun

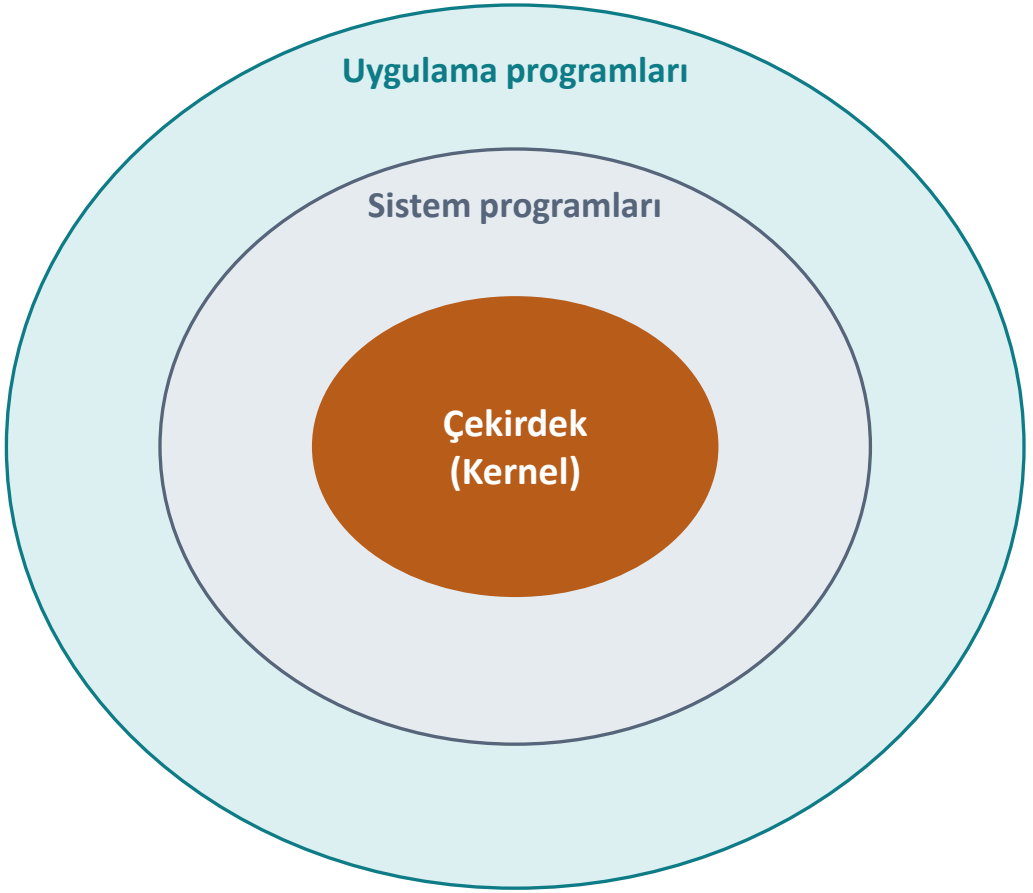
İşletim sistemi tek başına “yararlı iş” yapmaz; diğer programların yararlı iş yapabileceği ortamı sağlar.

İşletim Sistemi Nedir?

Çalışma tanımı

Bilgisayar donanımını yöneten ve kullanıcılar/uygulamalar ile donanım arasında aracılık eden yazılım.

- Evrensel olarak kabul görmüş tek bir tanım yoktur
- **Yaygın kabul: İS, bilgisayarda her zaman çalışan programdır → Çekirdek (Kernel)**
- Sistem programları İS ile birlikte gelir, ancak çekirdeğin parçası değildir
- Uygulama programları: sistemin işletilmesiyle ilgisi olmayan tüm programlar



İşletim Sistemine İki Bakış

Kullanıcı bakışı

“Benim için ne yapıyor?”

- Kişisel bilgisayar: kullanım kolaylığı öncelikli; kaynak kullanımı ikinci planda
- Paylaşılan büyük sistemler: kaynakların verimli ve adil kullanımı
- Mobil cihazlar: dokunmatik etkileşim, pil ömrü
- Gömülü sistemler: kullanıcı görünümü çok az ya da hiç yok

Sistem bakışı

“Donanım açısından görevi ne?”

- Kaynak ayırıcı (Resource Allocator): CPU zamanı, bellek, depolama ve G/Ç aygıtlarını dağıtır
- Çakışan isteklere verimli ve adil biçimde karar verir
- Denetim programı (Control Program): programların yürütülmesini denetler
- Hataları ve bilgisayarın yanlış kullanımını önler

Aynı İS, kime sorulduğuna göre farklı tanımlanır: kullanıcı kolaylık, sistem ise verimlilik ve denetim bekler.

İşletim Sistemi: Genişletilmiş Makine



Donanım düzeyinde disk

- Sektör ve iz adresleri
- Denetleyici komutları ve yazmaçları
- Motor, zamanlama, hata kodları

İS'nin sunduğu

Dosya

aç · oku · yaz · kapat

- Soyutlama (Abstraction), donanımın karmaşıklığını gizler
- Programcı disk geometrisiyle değil, dosyalarla çalışır
- Temel soyutlamalar — süreç, adres uzayı, dosya — sonraki haftaların konularıdır

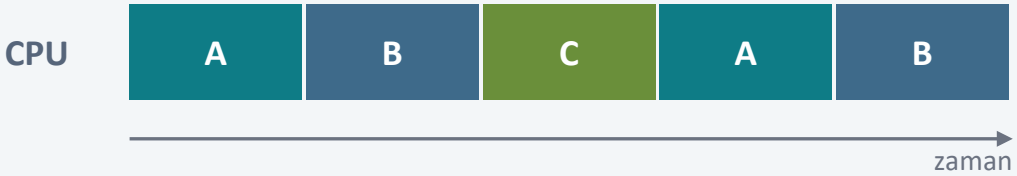
Kaynak: Tanenbaum & Bos, Bölüm 1.1.1 (şekil yeniden çizilmiştir)

İşletim Sistemi: Kaynak Yöneticisi

Üç program aynı anda yazıcıya çıktı gönderirse? Satırlar birbirine karışır. İS, çıktıları diskte tamponlayıp sırayla yazdırarak düzen sağlar.

Zamanda çoğullama

Kaynak sırayla kullanılır



- Örnek: CPU, yazıcı
- İS'nin sorusu: Sırada kim var, ne kadar süre kullanacak?

Uzayda çoğullama

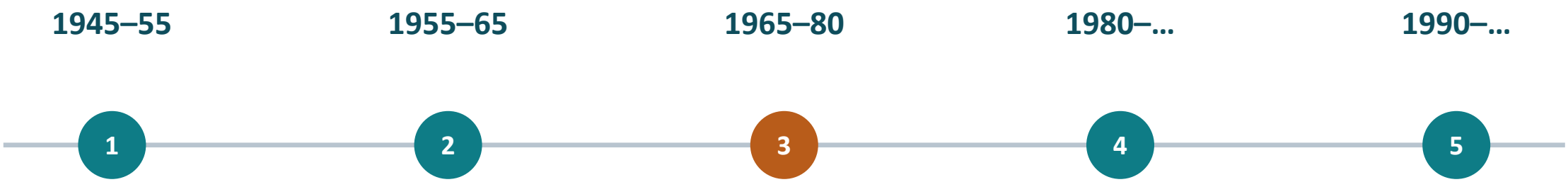
Kaynak parçalara bölünür



- Örnek: ana bellek, disk
- İS'nin sorusu: Kime ne kadar alan? Adalet ve koruma nasıl sağlanır?

Kaynak: Tanenbaum & Bos, Bölüm 1.1.2

İşletim Sistemlerinin Kısa Tarihi



Vakum tüpleri

İS yok; programlama doğrudan makine dilinde ya da kablolu panolarla yapılır

Transistörler ve toplu işlem

İşler gruplanır ve operatör beklemeden art arda çalıştırılır

Entegre devreler

Çoklu programlama, biriktirme (spooling) ve zaman paylaşımı

Kişisel bilgisayarlar

Tek kullanıcı; grafik arayüz ve kullanım kolaylığı öne çıkar

Mobil cihazlar

Dokunmatik etkileşim, enerji verimliliği, sürekli bağlantı

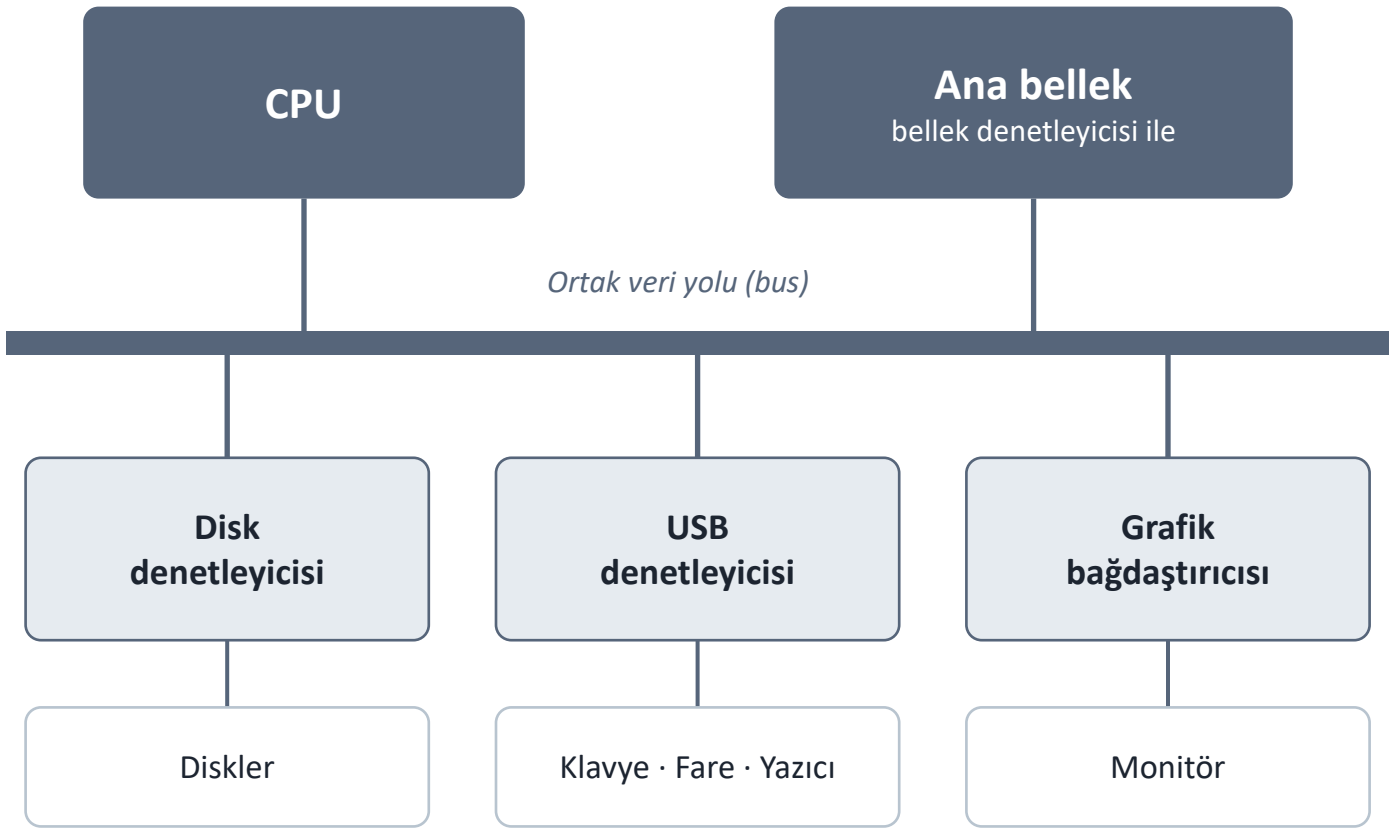
Dönemlendirme: Tanenbaum & Bos, Bölüm 1.2 · Çoklu programlama ve zaman paylaşımı Bölüm 5'te ayrıntılı ele alınacaktır

Her Yeni Yaklaşım Bir Sorunun Çözümüdür

SORUN	İS'NİN YANITI
Pahalı makine, işler arasında operatör ve hazırlık için boşta bekliyor	→ Toplu işlem (Batch): işler gruplanıp art arda çalıştırılır
Bir iş G/Ç beklerken CPU tamamen boşta kalıyor	→ Çoklu programlama: bellekte birden çok iş; CPU bekleyen işten diğerine geçer
Sonucu almak saatler sürüyor; programcı makineyle etkileşim kuramıyor	→ Zaman paylaşımı: her kullanıcı terminalinden etkileşimli çalışır
Donanım ucuzladı; bilgisayarı uzman olmayanlar da kullanıyor	→ Kişisel bilgisayar İS'leri: kullanım kolaylığı ve grafik arayüz
Cihazlar küçüldü, pille çalışıyor ve sürekli taşınıyor	→ Mobil İS'ler: enerji yönetimi, dokunmatik arayüz, sensörler

Kaynaklar: Tanenbaum & Bos, Bölüm 1.2; Silberschatz vd., Bölüm 1.4

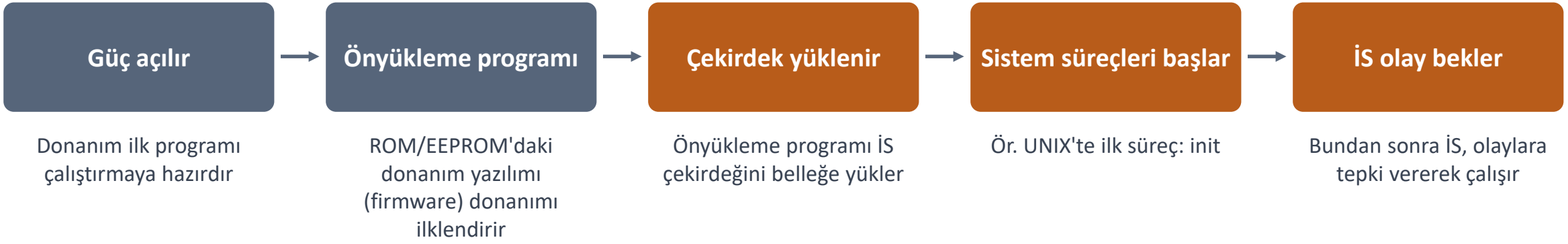
Modern Bir Bilgisayar Sisteminin Bileşenleri



- Her aygıt denetleyicisi belirli bir aygıt türünden sorumludur; yerel tamponu ve özel yazmaçları vardır
- CPU ve aygıt denetleyicileri paralel çalışır, bellek erişimi için yarışır
- Bellek denetleyicisi paylaşılan belleğe erişimi düzenler
- **İS, her denetleyiciyle bir aygıt sürücüsü (Device Driver) aracılığıyla konuşur**

Silberschatz vd., Şekil 1.2 temel alınarak yeniden çizilmiştir

İS ile Donanım: Açılıştan Olay Beklemeye



Olay = Kesme

Olaylar, donanımdan ya da yazılımdan gelen kesmelerle bildirilir. Bu nedenle modern işletim sistemleri kesme güdümlüdür (interrupt-driven).

İleri bağlantı

Önyükleme sürecinin ayrıntıları Hafta 2'de ele alınacaktır.

Kaynak: Silberschatz vd., Bölüm 1.2.1

CPU, Bir Olaydan Nasıl Haberdar Olur?

SEÇENEK A

Sürekli sormak (yoklama)

- CPU, döngü içinde aygıtın durumunu tekrar tekrar kontrol eder
- Olay hiç gerçekleşmese bile CPU zamanı harcanır

Benzetme: *Misafir geldi mi diye her dakika kapıya bakmak*

SEÇENEK B

Haber verilmesini beklemek (kesme)

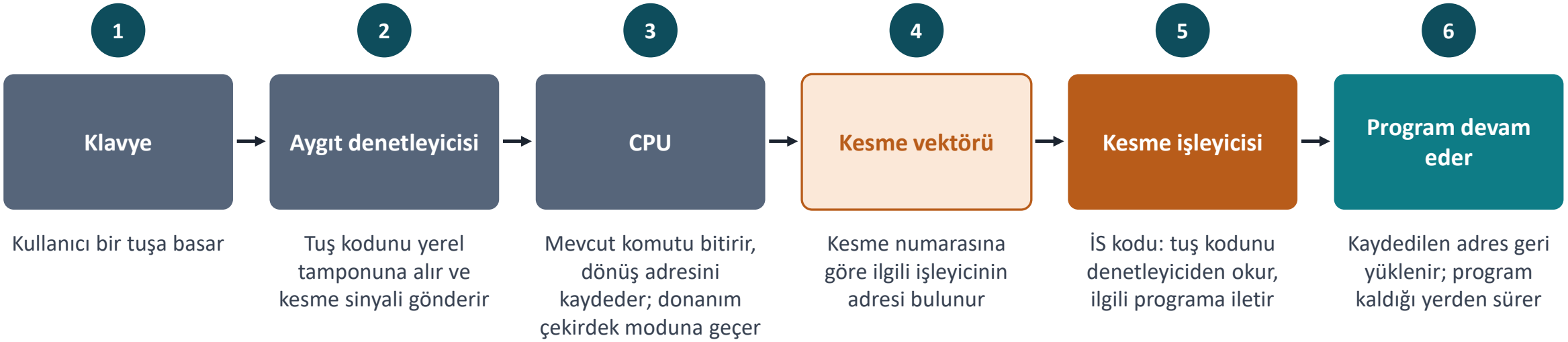
- Aygıt hazır olduğunda CPU'ya kendisi sinyal gönderir
- CPU bu arada başka işler yapabilir

Benzetme: *Kapı zili çalınca kapıya gitmek*

Kesme (Interrupt): Donanımın, genellikle sistem veri yolu üzerinden CPU'ya gönderdiği ve bir olayın gerçekleştiğini bildiren sinyal.

Yoklama ve kesme güdümlü G/Ç'nin ayrıntıları Hafta 11'de ele alınacaktır

Kesme Akışı: Klavyede Bir Tuşa Basıldığında



Kesme, CPU'nun o anki işini kısa süreliğine bırakıp İS'ye denetim vermesini sağlar; iş bitince kesilen program hiçbir şey olmamış gibi devam eder.

Kesme İşleme Adımları ve Kesme Vektörü

1. Aygıt, kesme isteği hattına sinyal verir
2. CPU her komuttan sonra bu hattı denetler ve kesmeyi fark eder
3. Kesilen programın dönüş adresi kaydedilir (modern mimarilerde sistem yığınına)
4. Kesme numarası ile kesme vektöründen işleyicinin adresi bulunur
5. Kesme hizmet yordamı çalışır; değiştireceği yazmaçları önce kaydeder, sonra geri yükler
6. Dönüş adresi program sayacına yüklenir; program kaldığı yerden devam eder

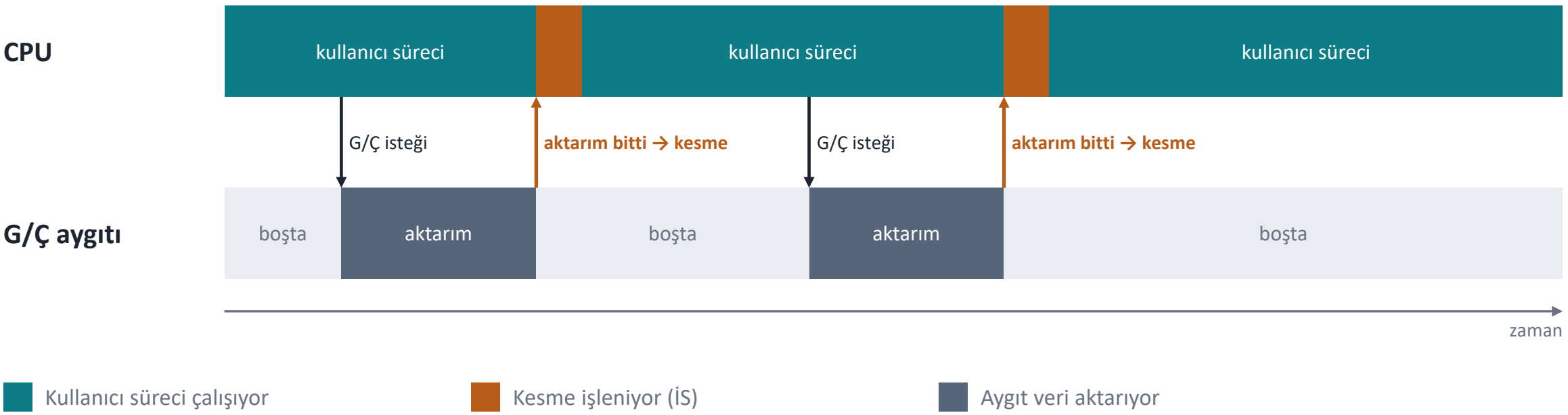
Kesme vektörü (temsilî)

No	İşleyici adresi
0	işleyici 0'ın adresi
1	işleyici 1'in adresi
2	işleyici 2'nin adresi
…	…
k	**klavye işleyicisinin adresi**

Kesme vektörü: kesme numarasıyla indekslenen ve kesme hizmet yordamlarının adreslerini tutan dizi.

Kaynak: Silberschatz vd., Bölüm 1.2.1 ve 13.2.2

Kesme Zaman Çizelgesi: CPU ile G/Ç Aygıtı Birlikte Çalışır



CPU ile G/Ç aygıtı paralel çalışır. Kesme, aktarımın bittiğini bildirerek ikisini eşgüdümler. Kesme işleme kısa sürer; CPU zamanının büyük kısmı kullanıcı sürecine kalır.

Silberschatz vd., Şekil 1.3 temel alınarak yeniden çizilmiştir

Kesme ve Tuzak (Trap / Exception)

Tuzak (Trap): Yazılımın ürettiği kesme. Bir hatadan (ör. sıfıra bölme, geçersiz bellek erişimi) ya da kullanıcı programının İS'den hizmet istemesinden kaynaklanır.

Ölçüt	Kesme (Interrupt)	Tuzak (Trap / Exception)
Kaynak	Donanım aygıtı (klavye, disk, Donanım Zamanlayıcısı…)	Çalışan programın yürüttüğü komut
Zamanlama	Programın akışından bağımsız gelir (asenkron)	Belirli bir komutun yürütülmesine bağlıdır (senkron)
Örnek	“Disk aktarımı tamamlandı”	Sıfıra bölme · geçersiz bellek erişimi · hizmet isteği
Ortak nokta	Her ikisinde de denetim, kesme vektörü üzerinden çekirdek modunda İS'ye geçer	

İleri bağlantı: Programın İS'den hizmet istemek için kullandığı tuzak (sistem çağrısı) Hafta 2'nin konusudur.

Depolama Yapısı: Ana Bellek ve İkincil Depolama

CPU komutları yalnızca ana bellekten yükleyebilir → çalışacak her program önce belleğe alınmalıdır.

Ana bellek (RAM)

- CPU'nun doğrudan erişebildiği tek büyük depolama alanı
- Genellikle DRAM ile gerçekleştirilir
- Uçucudur: güç kesilince içerik kaybolur
- Tüm program ve verileri kalıcı tutmak için çok küçüktür

yükleme ←

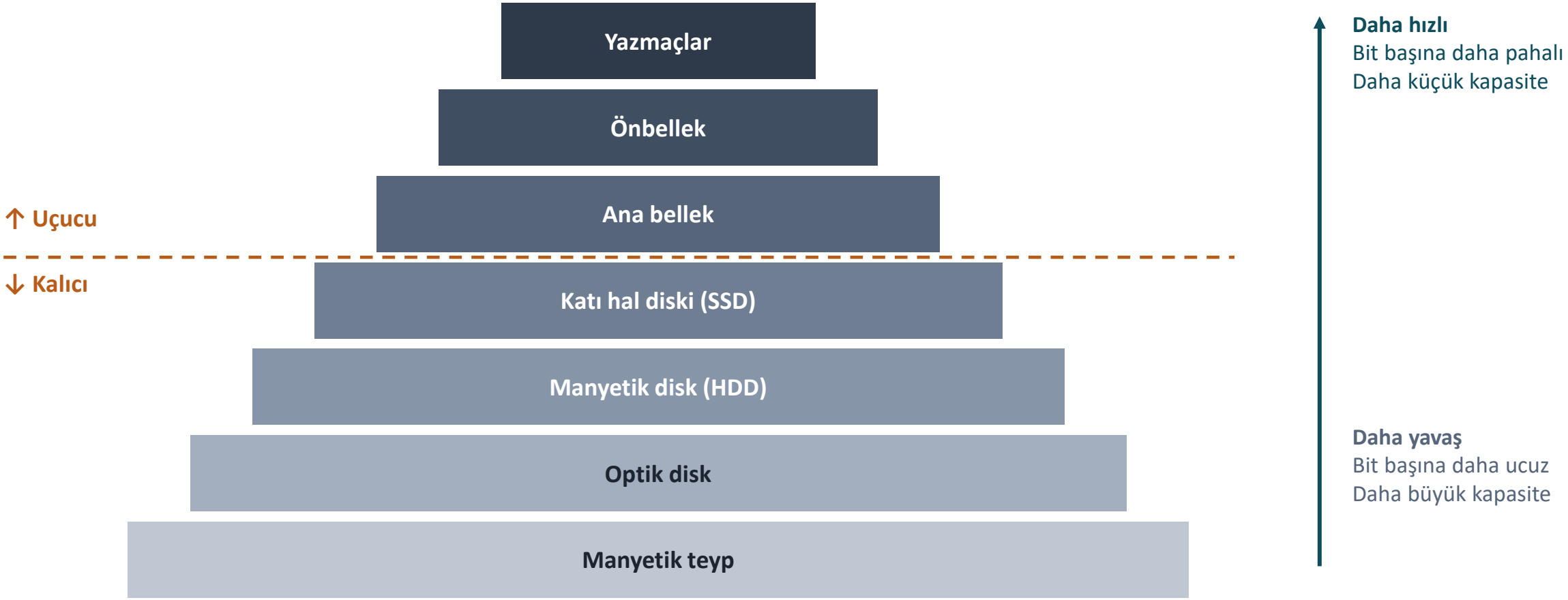
İkincil depolama

- Kalıcıdır: güç kesilse de veri korunur
- Büyük kapasite: HDD ve SSD
- Programlar ve veriler burada saklanır
- Ana bellekten çok daha yavaştır

İdeal bellek — hızlı, büyük, ucuz ve kalıcı — tek bir teknolojiyle mümkün değildir → depolama hiyerarşisi

Kaynak: Silberschatz vd., Bölüm 1.2.2

Depolama Hiyerarşisi



Silberschatz vd., Şekil 1.4 temel alınarak yeniden çizilmiştir · Hiyerarşi, hız–kapasite–maliyet ödünleşiminin sonucudur

Rakamlarla Depolama Hiyerarşisi

Düzey	Tipik boyut	Erişim süresi (ns)	Kim yönetir?
Yazmaçlar	< 1 KB	0,25 – 0,5	Derleyici
Önbellek	< 16 MB	0,5 – 25	Donanım
Ana bellek	< 64 GB	80 – 250	İşletim sistemi
Katı hal diski	< 1 TB	25.000 – 50.000	İşletim sistemi
Manyetik disk	< 10 TB	5.000.000	İşletim sistemi

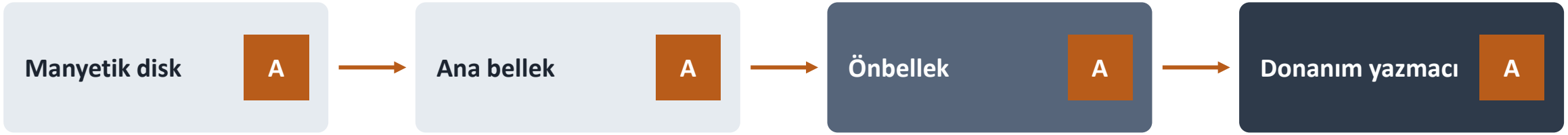
≈ 10 milyon kat

Manyetik disk erişimi (≈ 5.000.000 ns), yazmaç erişiminden (≈ 0,5 ns) yaklaşık on milyon kat daha yavaştır.

Bu fark, önbelleklemenin ve İS'nin depolama yönetiminin neden kritik olduğunu açıklar.

Değerler: Silberschatz vd., Şekil 1.11 (9. baskı, 2013). Tipik değerlerdir; güncel donanımda farklılık gösterebilir.

Önbellekleme (Caching)



Bir A verisinin diskten yazmaca yolculuğu: veri, kullanıldıkça daha hızlı düzeylere kopyalanır

Önbellekleme ilkesi

- Sık kullanılan veri geçici olarak daha hızlı bir depolamaya kopyalanır
- Veri gerektiğinde önce önbelleğe bakılır
- Varsa (isabet / hit): doğrudan kullanılır
- Yoksa (ıskalama / miss): yavaş düzeyden getirilir ve bir kopyası önbelleğe konur

Doğurduğu tasarım soruları

- Önbellek sınırlıdır → hangi veri tutulmalı, hangisi çıkarılmalı?
- Aynı veri birden çok düzeyde kopyalanır → kopyalar tutarlı kalmalı
- Çok işlemcili sistemlerde: önbellek tutarlılığı (Cache Coherency)

Silberschatz vd., Bölüm 1.8.3 ve Şekil 1.12 temel alınmıştır

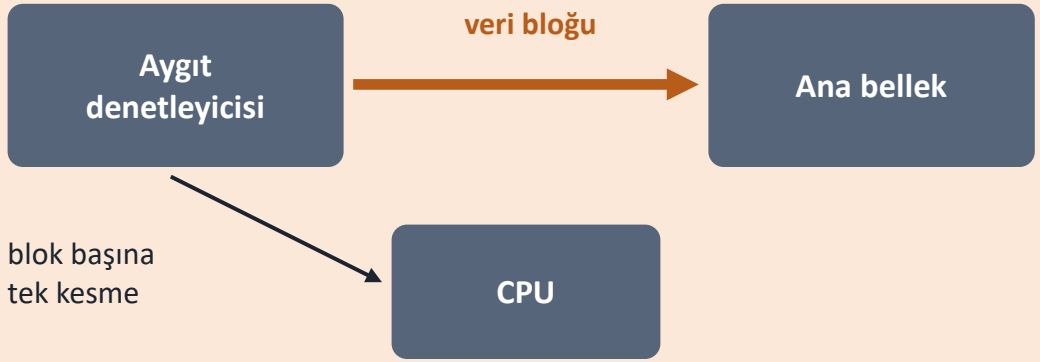
G/Ç Yapısı ve DMA'ya İlk Bakış

Kesme güdümlü G/Ç: küçük veri



her küçük birim için ayrı kesme

DMA: büyük veri blokları



Sorun

Hızlı aygıtlarda her küçük veri birimi için kesme üretmek, CPU'yu kesme işlemeye boğar.

Çözüm: Doğrudan Bellek Erişimi (DMA)

Denetleyici bloğu CPU'ya uğramadan bellek ile aygıt arasında aktarır; CPU bu sırada başka iş yapar.

Kaynak: Silberschatz vd., Bölüm 1.2.3 · DMA'nın adım adım işleyişi Hafta 11'de ele alınacaktır

Tek İşlemciden Çok İşlemciye

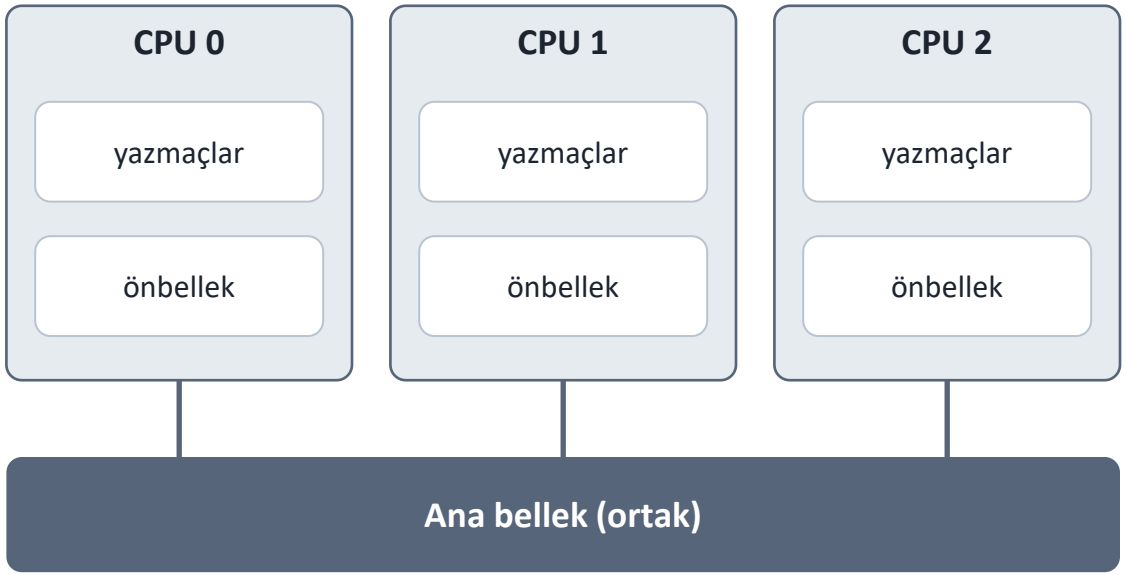
Tek işlemcili sistem

- Tek bir genel amaçlı CPU
- Disk, klavye gibi özel amaçlı işlemciler kullanıcı süreçlerini çalıştırmaz

Çok işlemcili sistemlerin üç avantajı

1. **Verim artışı:** Daha çok iş tamamlanır; ancak N işlemciyle hızlanma N'den azdır (eşgüdüm ek yükü)
2. **Ölçek ekonomisi:** Çevre birimleri, bellek ve güç kaynağı paylaşılır
3. **Güvenilirlik:** İşlevler işlemciler arasında uygun biçimde dağıtılmışsa, bir işlemcinin arızası sistemi tamamen durdurmayabilir; sistem daha düşük performansla çalışmaya devam edebilir.

Simetrik çok işlemcili (SMP) mimari



Asimetrik: bir patron işlemci diğerlerine görev dağıtır

Simetrik (SMP): tüm işlemciler eşittir; bugün yaygın yaklaşım

Kaynak: Silberschatz vd., Bölüm 1.3.1–1.3.2; SMP şeması Şekil 1.6 temel alınmıştır

Çok Çekirdekli (Multicore) Tasarım

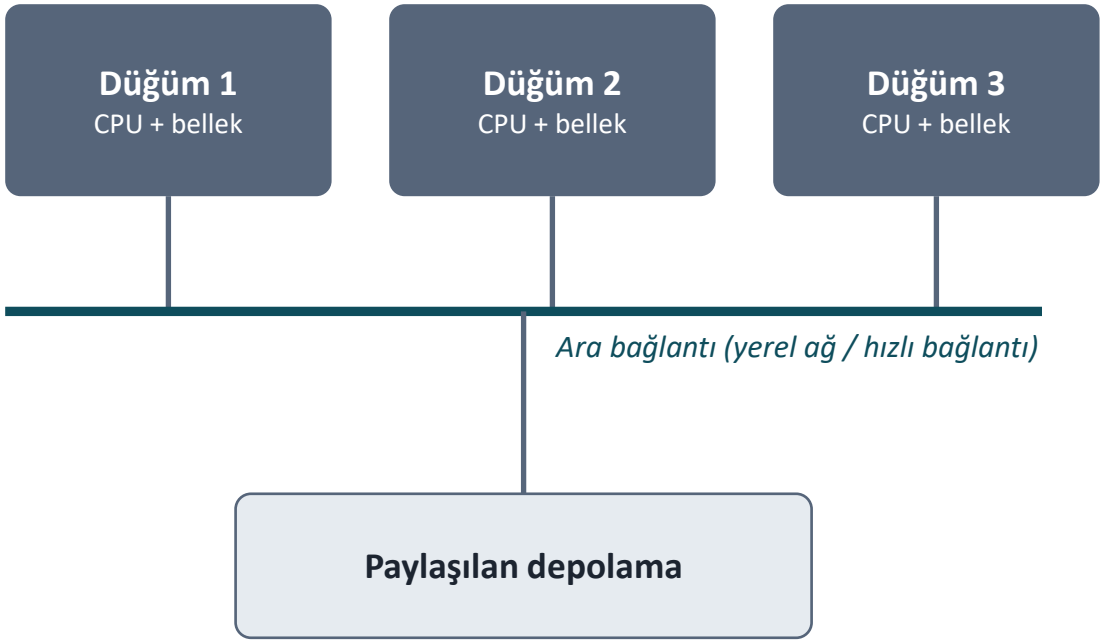


- Birden çok işlem çekirdeği tek bir yonga üzerindedir
- Yonga içi iletişim, yongalar arası iletişimden daha hızlıdır
- Birden çok tek çekirdekli yongaya göre daha az güç tüketir
- İS açısından N çekirdek, N ayrı işlemci gibi görünür

İleri bağlantı: Çok çekirdekten yararlanmak için eşzamanlı programlama gerekir → İş parçacıkları, Hafta 4

Kaynak: Silberschatz vd., Bölüm 1.3.2; şema Şekil 1.7 temel alınarak yeniden çizilmiştir

Kümelenmiş (Clustered) Sistemler



- İki veya daha fazla bağımsız sistemin (düğüm) bir araya gelmesi; genellikle depolama paylaşılır
- Amaç: yüksek erişilebilirlik — bir düğüm arızalanırsa diğeri işi devralır
- Asimetrik kümeleme: bir makine sıcak yedek (hot-standby) olarak yalnızca izler
- Simetrik kümeleme: tüm düğümler uygulama çalıştırır ve birbirini izler
- Yüksek başarımlı hesaplama: bir uygulama düğümlere bölünerek paralel çalıştırılır

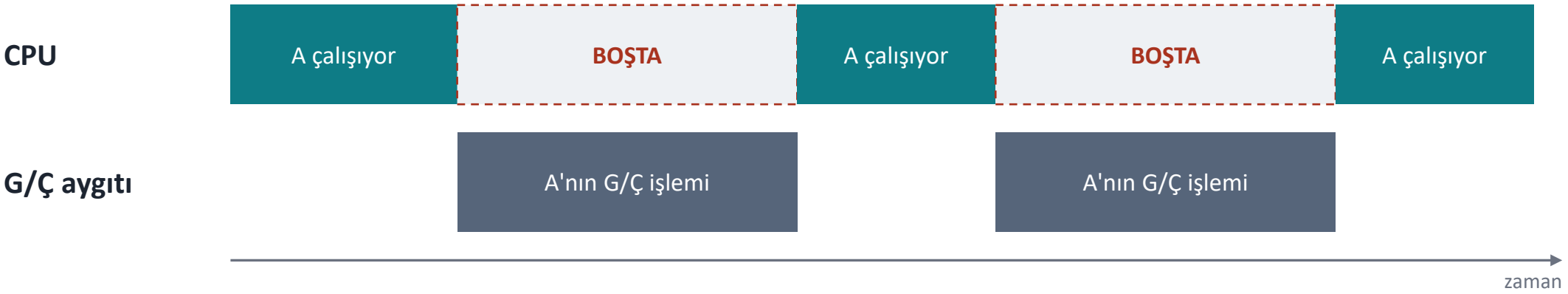
Kaynak: Silberschatz vd., Bölüm 1.3.3

Mimarilerin Karşılaştırması

Mimari	İşlem birimleri	Bellek	Temel kazanım	Tipik kullanım
Tek işlemcili	Tek genel amaçlı CPU	Tek bellek	Basitlik	Basit gömülü sistemler
Çok işlemcili (SMP)	Birden çok CPU	Ortak bellek	Verim, güvenilirlik	Sunucular, iş istasyonları
Çok çekirdekli	Tek yongada birden çok çekirdek	Ortak bellek	Hız ve düşük güç tüketimi	Dizüstü bilgisayarlar, akıllı telefonlar
Kümelenmiş	Ağla bağlı bağımsız bilgisayarlar	Her düğümün kendi belleği; depolama paylaşılabilir	Yüksek erişilebilirlik, ölçeklenebilirlik	Veri merkezleri, bilimsel hesaplama

Ortak soru: Birden çok işlem birimi olduğunda işler nasıl dağıtılacak ve paylaşılan veriler nasıl korunacak? → Hafta 5 ve Hafta 6–7

Sorun: Tek Bir Program CPU'yu Boşa Harcar



Gözlem

G/Ç işlemleri CPU'ya göre çok yavaştır (depolama hiyerarşisini hatırlayın). Tek bir program çalışırken CPU zamanının önemli kısmı beklemekle geçebilir.

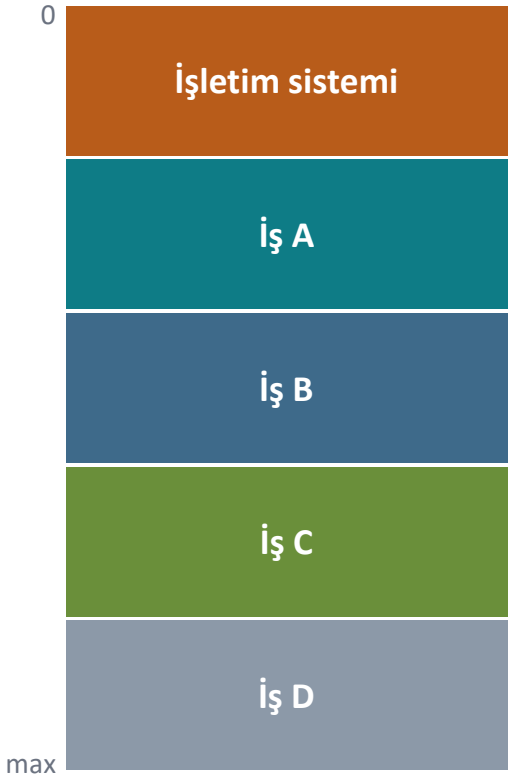
Soru

CPU beklerken bellekte çalışmaya hazır başka bir program olsaydı, CPU'yu ona veremez miydik?

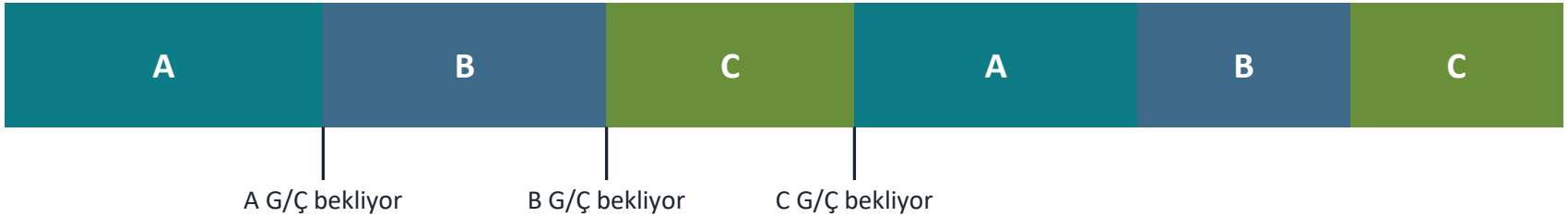
Şema kavramsaldır; süreler ölçekli değildir

Çoklu Programlama (Multiprogramming)

Bellek düzeni



CPU



Fikir

İS bellekte aynı anda birden çok iş tutar. Çalışan iş beklemeye geçince (ör. G/Ç), CPU bellekteki başka bir işe verilir. İşler önce diskteki iş havuzunda (job pool) bekler; bellek hepsine yetmez.

Sonuç: En az bir iş çalışmaya hazır olduğu sürece CPU boşta kalmaz → CPU kullanımı artar.

Silberschatz vd., Bölüm 1.4 ve Şekil 1.9 temel alınmıştır

Birden Fazla Program Bellekteyse Kaynakları Kim Yönetecek?

Bellekte aynı anda birden çok program

Diskteki işlerden hangisi belleğe alınacak?	Bellek programlar arasında nasıl paylaştırılacak?	CPU sıradaki hangi programa verilecek?	Programlar birbirine ve İS'ye zarar vermesin
↓	↓	↓	↓
İş zamanlaması	**Bellek yönetimi**	**CPU Zamanlayıcısı**	**Koruma**
Hafta 3	Hafta 9–10	Hafta 5	Bu hafta + Hafta 13

Çoklu programlama, işletim sisteminin temel görevlerinin neredeyse tamamını zorunlu kılar.

Zaman Paylaşımı (Time Sharing / Multitasking)

- Çoklu programlamanın mantıksal uzantısıdır
- CPU işler arasında çok sık geçiş yapar; kullanıcı programı çalışırken onunla etkileşir
- Yanıt süresi kısa olmalıdır: genellikle 1 saniyeden az
- Her kullanıcı, bilgisayarı yalnızca kendisi kullanıyormuş gibi algılar
- İnsan hızındaki G/Ç (ör. klavyeden yazma) bilgisayar için çok yavaştır; bu sırada CPU diğer kullanıcılara verilir



Bağlantı: Sık geçişlerde İS'nin denetimi düzenli olarak geri almasını sağlayan donanım, Donanım Zamanlayıcısıdır (Bölüm 6).

Kaynak: Silberschatz vd., Bölüm 1.4 · Şema kavramsaldır

Çoklu Programlama ile Zaman Paylaşımı Arasındaki Fark

Ölçüt	Çoklu programlama	Zaman paylaşımı
Temel amaç	CPU kullanımını artırmak	Hızlı yanıt ve etkileşim sağlamak
CPU ne zaman başka işe geçer?	Çalışan iş beklemeye geçtiğinde (ör. G/Ç)	Bekleme olmasa bile kısa aralıklarla, sık sık
Kullanıcı etkileşimi	Yok ya da sınırlı (toplu işler)	Var; kullanıcı programla doğrudan etkileşir
Öne çıkan ölçüt	Verim, CPU kullanımı	Yanıt süresi

Anahtar ilişki: Her zaman paylaşımlı sistem çoklu programlama kullanır; ancak her çoklu programlamalı sistem zaman paylaşımlı değildir.

Kullanıcı Programına Tam Donanım Yetkisi Verilirse?

Belleğe sınırsız erişim

Başka bir programın ya da İS'nin belleğine yazar; sistem çöker

Aygıtlara doğrudan erişim

Birden çok program aynı aygıta aynı anda yazar; veri bozulur

Kesmeleri kapatma

İS bir daha denetimi geri alamaz

Sonsuz döngü

Program CPU'yu hiç bırakmaz; diğer programlar çalışamaz

Gerçek örnek

MS-DOS, donanımsal ikili modu olmayan Intel 8088 için yazılmıştı. Hatalı bir program İS'nin üzerine yazabiliyor, birden çok program aynı aygıta aynı anda yazabiliyordu.

Çözüm

Yetki ayrımını donanım düzeyinde zorlamak

Kaynak: Silberschatz vd., Bölüm 1.5.1

İkili Mod: Kullanıcı Modu ve Çekirdek Modu

Kullanıcı modu

mod biti = 1

- Uygulama adına yürütme
- Ayrıcalıklı komutlar yürütülemez
- Bellek koruma mekanizmalarıyla birlikte, programın izin verilen adres alanı dışına erişmesi tuzağa yol açar.

Çekirdek modu

mod biti = 0

- İşletim sistemi adına yürütme
- Tüm komutlar yürütülebilir
- Diğer adları: denetçi (supervisor), sistem ya da ayrıcalıklı mod

TEMEL KURALLAR

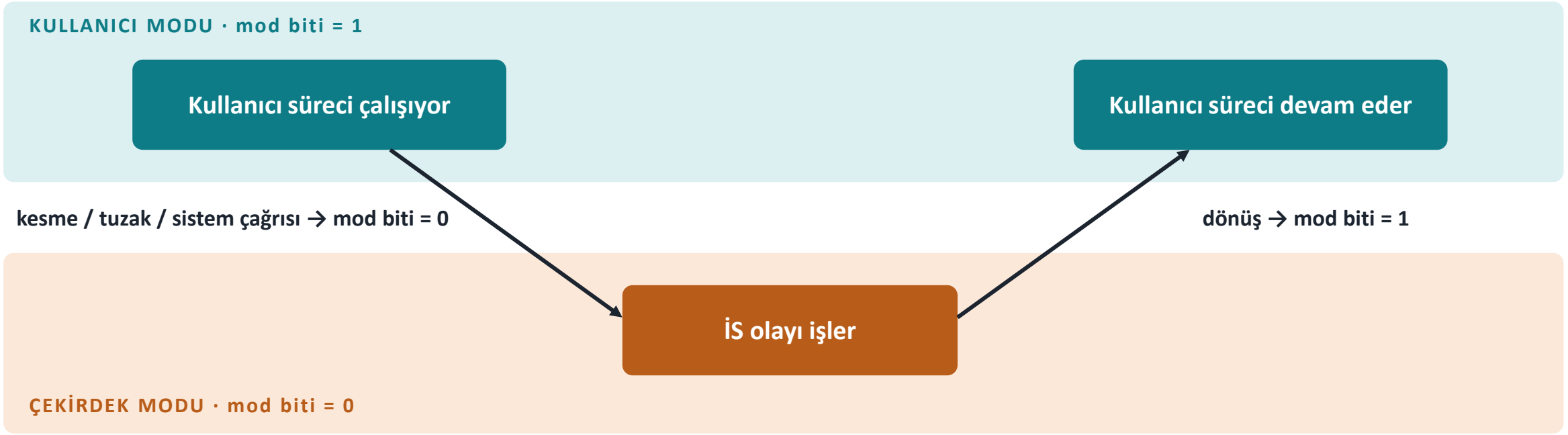
Sistem açılışta çekirdek modunda başlar

İS, kullanıcı programına geçmeden önce kullanıcı moduna geçer

Kesme veya tuzak olunca donanım otomatik olarak çekirdek moduna geçer

Mod biti donanımdadır. Kaynak: Silberschatz vd., Bölüm 1.5.1

Mod Geçişi: Kullanıcı Modundan Çekirdek Moduna ve Geri



Neden gerekli? Denetim İS'ye geçtiği her an sistem çekirdek modundadır; kullanıcı programına dönmeden önce ayrıcalık her zaman geri alınır. Böylece kullanıcı kodu hiçbir zaman ayrıcalıklı çalışmaz.

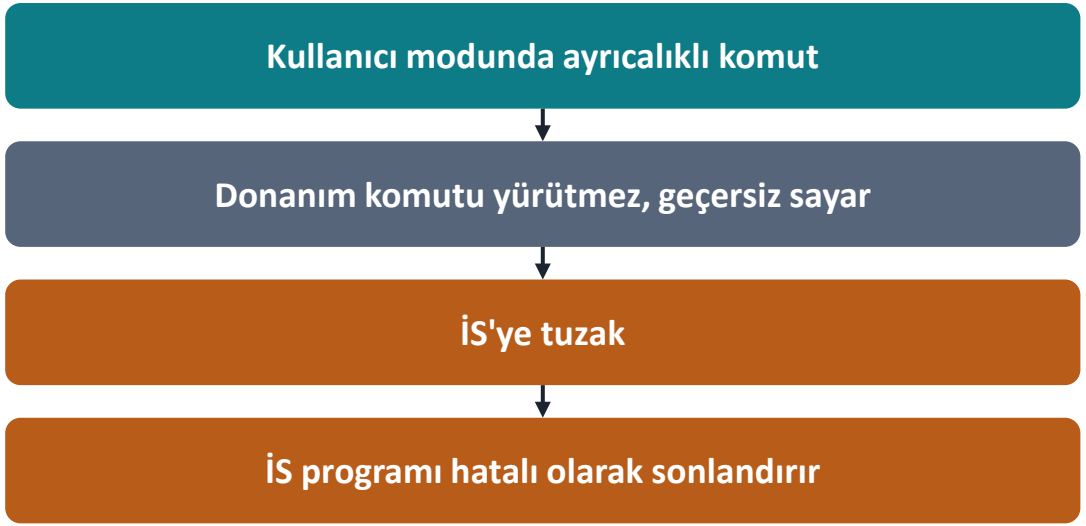
İleri bağlantı: Sistem çağrısının ayrıntıları Hafta 2'de.

Silberschatz vd., Şekil 1.10 temel alınarak yeniden çizilmiştir

Ayrıcalıklı Komutlar (Privileged Instructions)

Zarar verme potansiyeli taşıyan makine komutlarıdır.
Donanım bu komutları yalnızca çekirdek modunda yürütür.

Kullanıcı modunda denenirse:



Kaynak: Silberschatz vd., Bölüm 1.5.1

Örnekler

Çekirdek moduna geçiş komutu
kendi kendine yetki yükseltmeyi engeller

G/Ç denetimi
aygıtlara erişim İS üzerinden yapılır

Donanım Zamanlayıcısının yönetimi
İS'nin denetimi geri alması güvenceye alınır

Kesme yönetimi (ör. kesmeleri kapatma)
İS'nin olaylardan haberdar olması korunur

Bellek koruma yazmaçlarının değiştirilmesi
ayrıntısı Hafta 9'da

Uygulama: Hangi İşlemler Ayrıcalıklı Olmalı?

Aşağıdaki işlemlerden hangileri yalnızca çekirdek modunda yapılabilmelidir? Her biri için gerekçenizi söyleyin.

a Donanım Zamanlayıcısının değerini ayarlamak

b Saati okumak

c Belleği temizlemek

d Tuzak komutu vermek

e Kesmeleri kapatmak

f Aygıt durum tablosunu değiştirmek

g Kullanıcı modundan çekirdek moduna geçmek

h Bir G/Ç aygıtına erişmek

Silberschatz vd., Bölüm 1 alıştırmalarından (1.6) uyarlanmıştır

Çözüm: Ayrıcalıklı mı, Değil mi?

a AYRICALIKLI

Donanım Zamanlayıcısının değerini ayarlamak

Program kendi süresini sınırsız uzatabilir

b AYRICALIKSIZ

Saati okumak

Yalnızca okuma; sisteme zarar vermez

c AYRICALIKLI

Sistem belleğini temizlemek

İS'nin veya diğer programların belleğini etkileyebileceği için ayrıcalıklıdır.

d AYRICALIKSIZ

Tuzak komutu vermek

Kullanıcının İS'den hizmet isteme yoludur; denetimi zaten İS'ye verir

e AYRICALIKLI

Kesmeleri kapatmak

İS denetimi geri alamaz

f AYRICALIKLI

Aygıt durum tablosunu değiştirmek

İS'nin aygıtlar hakkındaki bilgisi bozulur

g AYRICALIKLI

Kullanıcı modundan çekirdek moduna geçmek

Aksi hâlde koruma tamamen anlamsızlaşır

h AYRICALIKLI

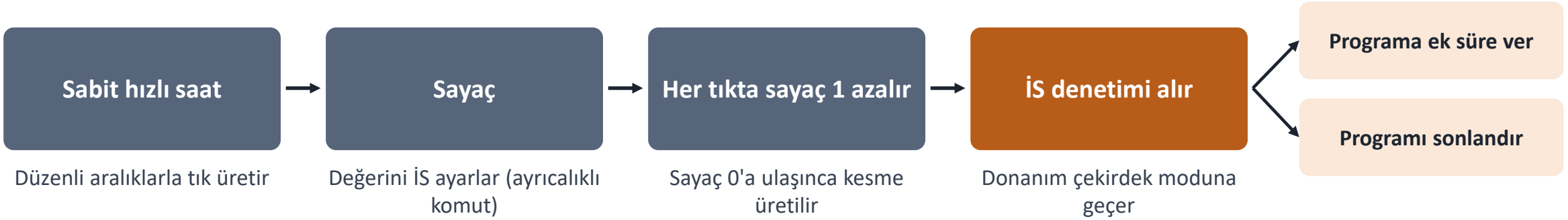
Bir G/Ç aygıtına erişmek

Aygıt paylaşımı ve veri bütünlüğü İS denetiminde olmalı

Genel ilke: Sistemin bütününü ya da başka programları etkileyebilecek her işlem ayrıcalıklıdır.

Donanım Zamanlayıcısı (Timer)

Sorun: Sonsuz döngüye giren ya da hiç sistem hizmeti çağırmayan bir program CPU'yu İS'ye hiç geri vermezse?



Sayısal örnek

10 bitlik sayaç ve 1 ms'lik saat → kesme aralığı 1 ms ile 1.024 ms arasında, 1 ms'lik adımlarla ayarlanabilir.

Kural

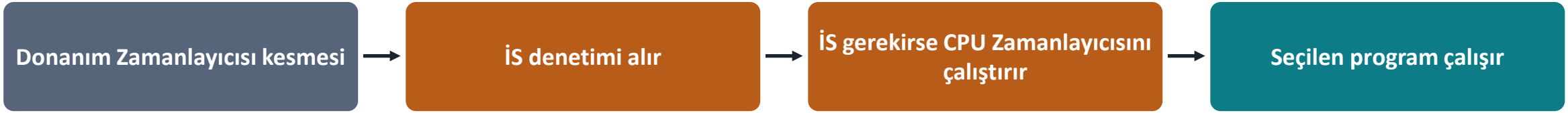
İS, denetimi kullanıcı programına vermeden önce Donanım Zamanlayıcısını kurar. Zamanlayıcıyı değiştiren komutlar ayrıcalıklıdır.

Kaynak: Silberschatz vd., Bölüm 1.5.2

Donanım Zamanlayıcısı ile CPU Zamanlayıcısı Aynı Şey Değildir

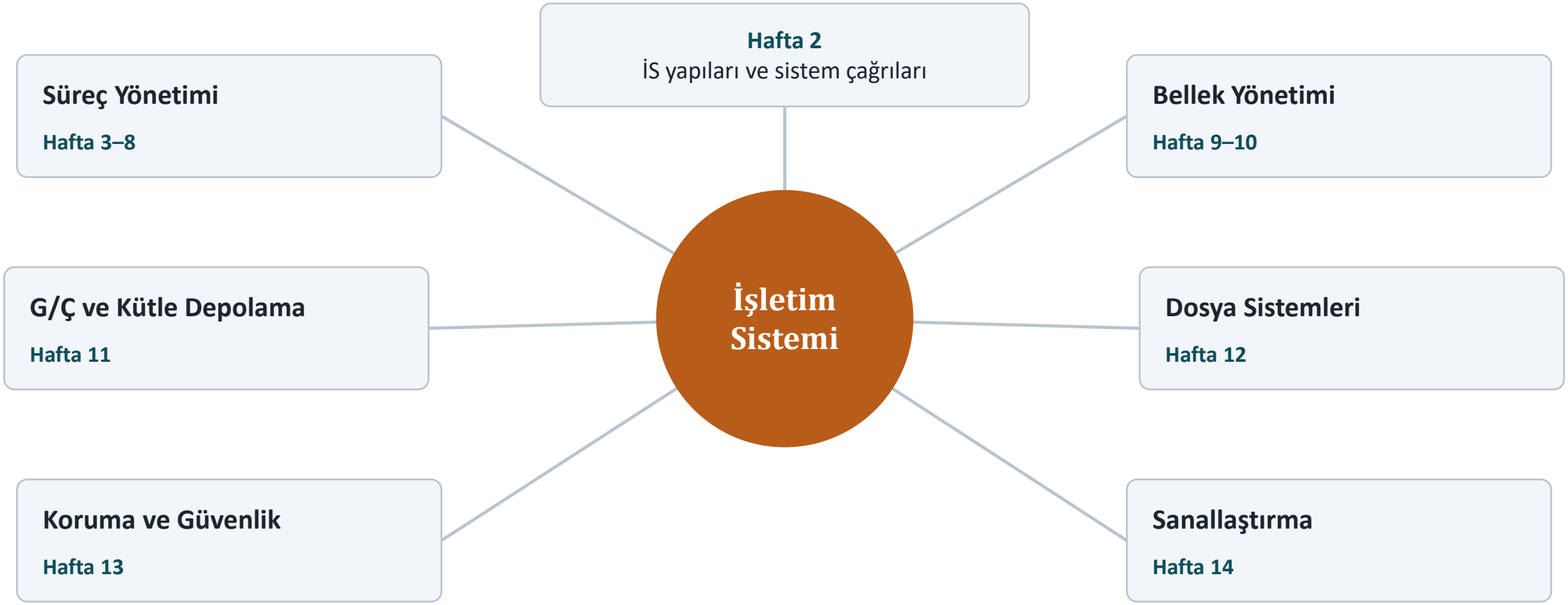
Ölçüt	Donanım Zamanlayıcısı (Timer)	CPU Zamanlayıcısı (CPU Scheduler)
Türü	Donanım bileşeni	İS'nin bir yazılım bileşeni
Görevi	Belirlenen süre dolunca kesme üretmek	Hazır programlardan CPU'yu kime vereceğine karar vermek
Karar verir mi?	Hayır — yalnızca “süre doldu” sinyali verir	Evet — “sırada kim var?” sorusunu yanıtlar
Ders	Bu hafta	Hafta 5

İKİSİ NASIL BİRLİKTE ÇALIŞIR?



Benzetme: Donanım Zamanlayıcısı bir alarm saatidir; CPU Zamanlayıcısı ise karar vericidir.

İşletim Sisteminin Görevleri: Dönemin Yol Haritası



Her hafta, bu haritadaki bir İS görevine odaklanır.

Her Alanda İşletim Sisteminin Sorumlulukları

Süreç yönetimi

- Süreç oluşturma ve silme
- Süreçleri askıya alma ve sürdürme
- Senkronizasyon, iletişim ve kilitlenme yönetimi mekanizmaları

Bellek yönetimi

- Belleğin hangi kısmını kimin kullandığını izleme
- Belleğe hangi süreç ve verilerin alınacağına karar verme
- Bellek ayırma ve geri alma

Depolama yönetimi

- Dosya sistemi: dosya ve dizin oluşturma, silme
- Kütle depolama: boş alan yönetimi, ayırma, disk zamanlama
- Önbellekleme

G/Ç yönetimi

- Aygıt sürücüleri aracılığıyla aygıtları denetleme
- Tamponlama, önbellekleme ve biriktirme (spooling)
- Donanım ayrıntılarını kullanıcıdan gizleme

Koruma ve güvenlik

- Koruma: süreç ve kullanıcıların kaynaklara erişimini denetleme
- Güvenlik: sistemi iç ve dış saldırılara karşı savunma
- Kullanıcı kimlikleri (User ID) ile yetki ayrımı

Kaynak: Silberschatz vd., Bölüm 1.6–1.9

Hesaplama Ortamları

1 Geleneksel

- Masaüstü ve dizüstü bilgisayarlar
- Ağ üzerinden sunuculara erişim; ev ve ofis ağları

2 Mobil

- Akıllı telefon ve tabletler; GPS, ivmeölçer gibi sensörler
- Sınırlı güç ve bellek · iOS, Android

3 İstemci–sunucu

- Sunucular, istemcilerin isteklerini karşılar
- Eşler arası (P2P): her düğüm hem istemci hem sunucu olabilir

4 Bulut

- Hesaplama, depolama ve uygulamaların ağ üzerinden hizmet olarak sunulması
- Sanallaştırmaya dayanır

5 Gömülü

- Araç motorları, üretim robotları, ev aletleri
- Belirli bir görev; çoğunlukla sınırlı arayüz

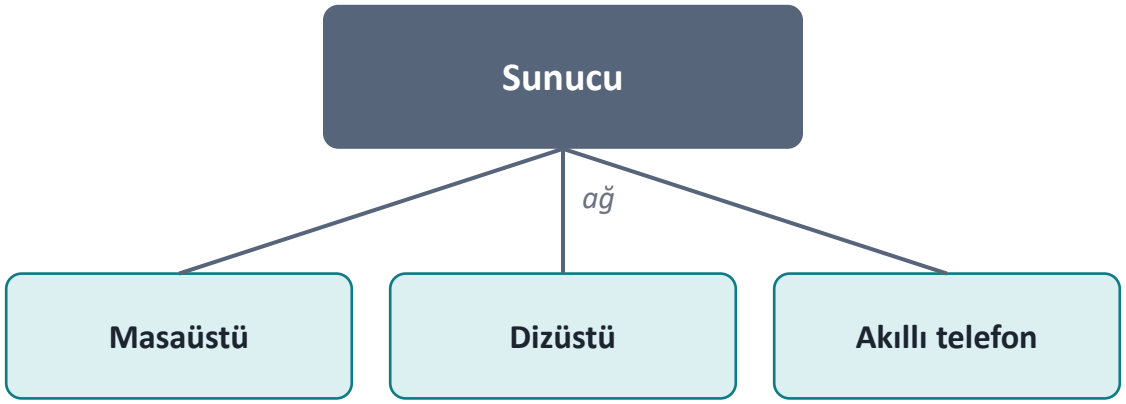
6 Gerçek zamanlı

- Katı ve iyi tanımlanmış zaman kısıtları
- Sonuç ancak zamanında üretilirse doğrudur

Kaynak: Silberschatz vd., Bölüm 1.11

İstemci–Sunucu ve Bulut Bilişim

İstemci–sunucu modeli



Hesaplama sunucusu: istenen işlemi yapar, sonucu döndürür

Dosya sunucusu: dosya oluşturma, güncelleme, okuma ve silme arayüzü sunar

P2P: istemci–sunucu ayrımı yoktur

Bulut bilişim

Dağıtım türleri

Genel	Özel	Karma

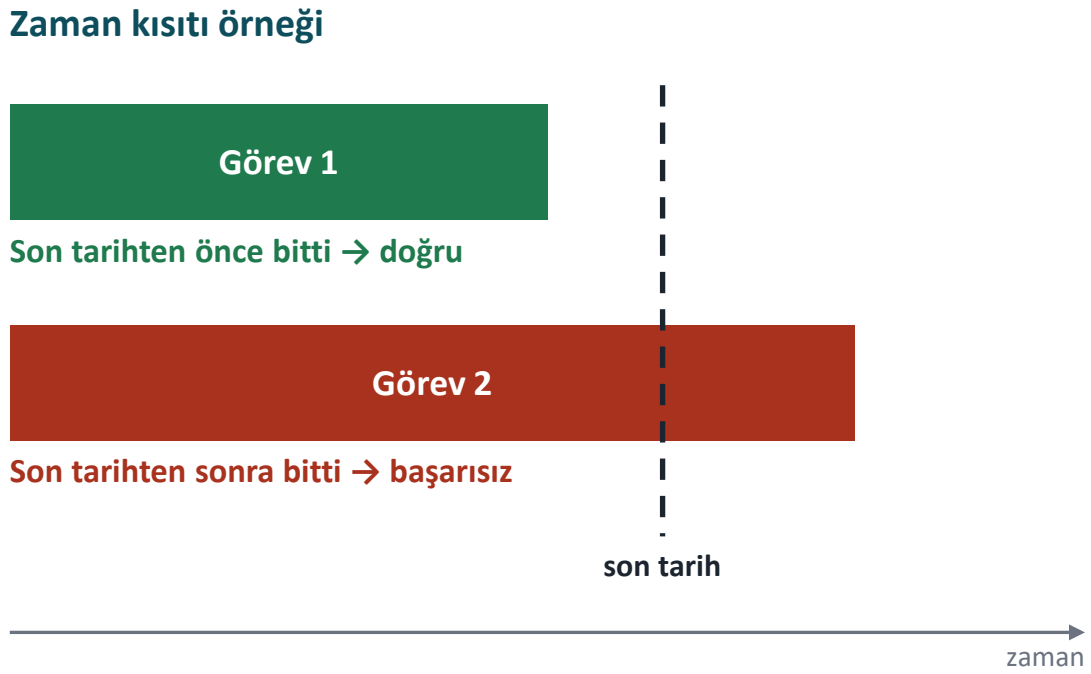
Hizmet modelleri

SaaS	Uygulama (ör. kelime işlemci, e-tablo)
PaaS	Kullanıma hazır yazılım yığını (ör. veritabanı sunucusu)
IaaS	Sunucu ve depolama altyapısı

Kaynak: Silberschatz vd., Bölüm 1.11 · Sanallaştırma ve bulut altyapısı Hafta 14'te ele alınacaktır

Gömülü ve Gerçek Zamanlı Sistemler

- Gömülü bilgisayarlar, var olan en yaygın bilgisayar türüdür
- Genellikle tek bir görev için tasarlanır; kullanıcı arayüzü çok sınırlıdır
- Çoğu, gerçek zamanlı bir İS çalıştırır
- **Gerçek zamanlı sistem: işlem, belirlenen zaman kısıtı içinde tamamlanmazsa sistem başarısız olur**



Zaman paylaşımı: hızlı yanıt istenir, ancak zorunlu değildir

Gerçek zamanlı: zaman kısıtına uymak doğruluğun parçasıdır

Kaynak: Silberschatz vd., Bölüm 1.11.8 · Örnekler: araç motorları, endüstriyel robotlar, tıbbi görüntüleme sistemleri

Açık Kaynaklı İşletim Sistemleri

Kapalı kaynak — *Ör. Microsoft Windows*

Yalnızca derlenmiş ikili kod dağıtılır

Karma yaklaşım — *Ör. macOS, iOS*

Açık kaynaklı Darwin çekirdeği + kapalı bileşenler

Açık kaynak — *Ör. Linux, BSD UNIX*

Kaynak kodu incelenebilir ve değiştirilebilir

Linux nasıl doğdu?

GNU projesi — Derleyici, düzenleyici gibi UNIX uyumlu araçlar üretti; ancak çekirdek yayımlamadı

1991 — Finlandiya'da bir öğrenci olan Linus Torvalds, GNU araçlarıyla ilkel bir UNIX benzeri çekirdek yayımladı

İnternet — Dünyanın dört bir yanından geliştiricilerin katkısıyla sistem hızla büyüdü

Özgür yazılım: FSF'nin GNU GPL lisansı “copyleft” ilkesini uygular: kaynak kodu paylaşılır ve değiştirilmiş sürümler de aynı koşullarla dağıtılır.

Kaynak: Silberschatz vd., Bölüm 1.12

Bu Hafta Neler Öğrendik?

1. Donanım kaynakları sınırlıdır ve paylaşılmalıdır
2. İS: genişletilmiş makine ve kaynak yöneticisi
3. Olaylar kesmelerle bildirilir; İS kesme güdümlüdür
4. Hız farkları → depolama hiyerarşisi, önbellekleme, DMA
5. CPU boş kalmasın → çoklu programlama, zaman paylaşımı
6. Paylaşım koruma ister → ikili mod, ayrıcalıklı komutlar, Donanım Zamanlayıcısı

Gelecek hafta: Kullanıcı modundaki bir program İS'den hizmeti nasıl ister? → İS hizmetleri, sistem çağrıları ve İS yapıları

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“İşletim sistemi = masaüstü ve pencereler”	Arayüz yalnızca görünen yüzdür; asıl kaynak yönetimi çekirdekte yapılır
“Donanım Zamanlayıcısı ile CPU Zamanlayıcısı aynıdır”	Biri kesme üreten donanımdır, diğeri CPU'yu kime vereceğine karar veren İS yazılımıdır
“Çoklu programlama = zaman paylaşımı”	Zaman paylaşımı, çoklu programlamanın etkileşim odaklı uzantısıdır
“Çekirdek modu = yönetici (root) hesabı”	Mod bir donanım durumudur; yönetici hesabının programları da kullanıcı modunda çalışır
“Kesme bir hata olduğunu gösterir”	Kesmelerin çoğu olağan olaylardır: G/Ç tamamlandı, tuşa basıldı, süre doldu

Kontrol Soruları

KAVRAMSAL

S1 **İşletim sisteminin “genişletilmiş makine” ve “kaynak yöneticisi” rollerini birer örnekle açıklayınız.**

İlgili bölüm: 1 · İşletim sistemi nedir?

S2 **Kesme ile tuzak arasındaki farkı kaynak ve zamanlama açısından açıklayınız.**

İlgili bölüm: 3 · Kesmeler

Kontrol Soruları

KARŞILAŞTIRMA

S3 **Çoklu programlama ile zaman paylaşımını amaç, CPU'nun başka işe geçtiği an ve kullanıcı etkileşimi açısından karşılaştırınız.**

İlgili bölüm: 5 · Çoklu programlama ve zaman paylaşımı

S4 **Simetrik çok işlemcili bir sistem ile kümelenmiş bir sistem arasındaki temel fark nedir? Hangisi yüksek erişilebilirlik için tasarlanır?**

İlgili bölüm: 4 · Sistem mimarileri

Kontrol Soruları

KISA SENARYO

S5 **Bir kullanıcı programına G/Ç aygıtlarını doğrudan kontrol etme izni verilirse ne tür problemler ortaya çıkabilir?**

İpucu: Aynı aygıtı aynı anda kullanmak isteyen iki programı düşünün.

S6 **Sistemde Donanım Zamanlayıcısı bulunmasaydı, sonsuz döngüye giren bir program ne gibi sonuçlara yol açardı?**

İpucu: İS denetimi hangi yollarla geri alabilir?

Kontrol Soruları

NEDEN–SONUÇ

S7 **Önbellekleme neden işe yarar? Önbelleğin boyutunun sınırlı olması hangi yeni tasarım sorusunu doğurur?**

İlgili bölüm: 3 · Depolama hiyerarşisi ve önbellekleme

S8 **Donanımında mod biti olmayan bir işlemci üzerinde güvenilir bir çok kullanıcılı İS yazmak neden zordur?**

İpucu: MS-DOS ve Intel 8088 örneğini hatırlayın.

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 1 — 1.1–1.9, 1.11, 1.12

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 1 — 1.1–1.5

Not: Bu sunumdaki diyagramlar, belirtilen yerlerde kaynak kitaplardaki şekiller temel alınarak yeniden çizilmiştir.